STUDENT PRACTICE PACK

Pandas Data Pipelines and FastAPI Applications

Hands-on preparation workbook • Problem statements, data, requirements, and test checklists

How to use this pack

- Complete the exercises in order. Write and run the code yourself before checking the expected behavior.
- For Pandas, keep each transformation in its own function and pass the returned DataFrame to the next function.
- For FastAPI, build the database layer, ORM models, Pydantic schemas, routes, and tests as separate files.
- Do not hard-code sample records or expected summary values. The code should work when records change.
- The CSV files are supplied separately in the accompanying ZIP, and their contents are also shown in this document.
- Use the final checklists to test normal cases, invalid input, missing records, duplicates, and relationship boundaries.

Contents

1. Exercise 1 — Training Enquiry Fee Summary (Pandas)
2. Exercise 2 — Customer Service Request Summary (Pandas)
3. Exercise 3 — Consultant and Practice Area API (FastAPI)
4. Exercise 4 — Trainer and Skill Group API (FastAPI)
5. Appendix — Setup, run commands, and self-checklists

Exercise 1 — Training Enquiry Fee Summary (Pandas)

A training institute needs a report showing the average discounted fee and number of valid enquiries for each course. Raw enquiry data contains missing emails, duplicate enquiry IDs, and a reference ID that may not exist in the course catalogue.

Files and structure

```
Project/
|– main.py
|– requirements.txt
|– tests.py
`– data/
   |--training_enquiries.csv
   `--training_courses.csv
```

Dependencies: pandas and pytest.

Input data

training_enquiries.csv

enquiry_id	course_id	student_name	email	phone	quoted_fee	discount_pct	enquiry_date
3001	11	Aarav Rao	aarav@example.com	9876501011	30000	10	2026-09-01
3002	12	Diya Sen	diya@example.com	9876501012	24000	5	2026-09-02
3003	13	Kabir Das	kabir@example.com	9876501013	20000	0	2026-09-03
3004	14	Meera Nair		9876501014	18000	10	2026-09-04
3005	15	Rohan Patil	rohan@example.com	9876501015	22000	5	2026-09-05
3006	11	Sana Ali	sana@example.com	9876501016	32000	15	2026-09-06
3007	14	Vikram Roy	vikram@example.com	9876501017	19000	0	2026-09-07
3002	12	Diya Sen Duplicate	diya2@example.com	9876501099	25000	0	2026-09-08
3008	99	Nisha Paul	nisha@example.com	9876501018	26000	10	2026-09-09

training_courses.csv

course_id	course_name	category	duration_months	standard_fee
11	Data Analytics	Data	4	35000
12	Python Full Stack	Software	6	30000
13	Cloud Fundamentals	Cloud	3	25000
14	Software Testing	Quality	3	22000
15	Generative AI Basics	AI	2	28000

Required implementation

Implement all six functions in main.py. Each function must return a DataFrame. Do not print from pipeline functions.

load_student_enquiries(file_path)

Read training_enquiries.csv. Return exactly: enquiry_id, course_id, student_name, email, phone, quoted_fee, discount_pct. Exclude enquiry_date.

clean_enquiry_data(df)

Remove rows with missing email. For duplicate enquiry_id values, keep the first occurrence in input order.

merge_with_courses(enquiries_df, ref_file_path)

Read training_courses.csv; inner-join on course_id. Remove unmatched records. Keep all enquiry columns plus course_name, category, duration_months, standard_fee.

add_net_quoted_fee(df)

Add net_quoted_fee = quoted_fee * (1 - discount_pct / 100), rounded to two decimal places.

aggregate_by_course(df)

Return one row per course with course_name, avg_net_quoted_fee (mean net_quoted_fee), and enquiry_count (number of valid enquiry rows). Reset the index.

finalize_summary(df)

Return columns in this order: course_name, avg_net_quoted_fee, enquiry_count. Round average to two decimals, sort alphabetically by course_name, and reset index.

Main block

Under if __name__ == '__main__':, call all six functions in the specified order, passing each result to the next step. Print only the final summary DataFrame. Keep file paths in parameters or in the main block.

Constraints

- Use vectorized Pandas operations; do not use row-by-row loops inside pipeline functions.
- Do not modify input DataFrames in place.
- Do not keep intermediate DataFrames in global variables.
- Do not hard-code course/service names, counts, averages, fees, or individual row values.
- Function signatures and parameter names must match this exercise's specification.

Test checklist

- A missing email row is removed.
- A repeated ID keeps the first row, not the last.
- A record with an unknown reference ID is removed by the inner join.
- The date column is not returned by the loading function.
- Net fees are rounded to two decimal places.
- Aggregation counts valid joined records and computes the mean of net fees.
- The final output has the exact column order, alphabetical sorting, and a clean index.
- Importing main.py does not run the pipeline.

Exercise 2 — Customer Service Request Summary (Pandas)

A technology support company wants a summary of quoted service fees after discounts. Its service request data may contain missing customer emails, repeated request IDs, and requests for services absent from the catalogue.

Files and structure

```
Project/
|– main.py
|– requirements.txt
|– tests.py
`– data/
   |--service_requests.csv
   `--service_catalogue.csv
```

Dependencies: pandas and pytest.

Input data

service_requests.csv

request_id	service_id	customer_name	email	phone	quoted_fee	discount_pct	request_date
4101	21	Home Wi-Fi Setup	customer1@example.com	9000011001	1200	10	2026-09-11
4102	22	Laptop Repair		9000011002	2500	5	2026-09-12
4103	23	Data Backup	customer3@example.com	9000011003	1800	0	2026-09-13
4104	24	Printer Setup	customer4@example.com	9000011004	900	10	2026-09-14
4105	21	Home Wi-Fi Setup	customer5@example.com	9000011005	1400	15	2026-09-15
4106	25	Cybersecurity Check	customer6@example.com	9000011006	3200	5	2026-09-16
4107	22	Laptop Repair	customer7@example.com	9000011007	2300	0	2026-09-17
4103	23	Data Backup Duplicate	duplicate@example.com	9000011099	1900	0	2026-09-18
4108	88	Unknown Service	customer8@example.com	9000011008	700	0	2026-09-19

service_catalogue.csv

service_id	service_name	category	duration_days	standard_fee
21	Home Wi-Fi Setup	Networking	1	1500
22	Laptop Repair	Hardware	2	3000
23	Data Backup	Data Services	1	2200
24	Printer Setup	Peripherals	1	1200
25	Cybersecurity Check	Security	2	4000

Required implementation

Implement all six functions in main.py. Each function must return a DataFrame. Do not print from pipeline functions.

load_service_requests(file_path)

Read service_requests.csv. Return exactly: request_id, service_id, customer_name, email, phone, quoted_fee, discount_pct. Exclude request_date.

clean_service_requests(df)

Remove rows with missing email. For duplicate request_id values, keep the first occurrence in input order.

merge_with_services(requests_df, ref_file_path)

Read service_catalogue.csv; inner-join on service_id. Remove unmatched records. Keep all request columns plus service_name, category, duration_days, standard_fee.

add_net_quoted_fee(df)

Add net_quoted_fee = quoted_fee * (1 - discount_pct / 100), rounded to two decimal places.

aggregate_by_service(df)

Return one row per service with service_name, avg_net_quoted_fee (mean net_quoted_fee), and request_count (number of valid request rows). Reset the index.

finalize_summary(df)

Return columns in this order: service_name, avg_net_quoted_fee, request_count. Round average to two decimals, sort alphabetically by service_name, and reset index.

Main block

Under if __name__ == '__main__':, call all six functions in the specified order, passing each result to the next step. Print only the final summary DataFrame. Keep file paths in parameters or in the main block.

Constraints

- Use vectorized Pandas operations; do not use row-by-row loops inside pipeline functions.
- Do not modify input DataFrames in place.
- Do not keep intermediate DataFrames in global variables.
- Do not hard-code course/service names, counts, averages, fees, or individual row values.
- Function signatures and parameter names must match this exercise's specification.

Test checklist

- A missing email row is removed.
- A repeated ID keeps the first row, not the last.
- A record with an unknown reference ID is removed by the inner join.
- The date column is not returned by the loading function.
- Net fees are rounded to two decimal places.
- Aggregation counts valid joined records and computes the mean of net fees.
- The final output has the exact column order, alphabetical sorting, and a clean index.
- Importing main.py does not run the pipeline.

Exercise 3 — Consultant and Practice Area API (FastAPI)

A consulting organization manages practice areas and the consultants assigned to them. Administrators create practice areas, add consultants, retrieve a practice area with its consultants, and update a consultant's details. Every consultant belongs to one practice area.

Project structure

```
Project/
|– database.py
|– models.py
|– schemas.py
|– main.py
|– requirements.txt
`– tests.py
```

SQLite database: consulting.db. Required dependencies: fastapi, uvicorn, sqlalchemy, pydantic, pytest, and httpx for API tests.

Database and relationship

Create a Practice Area table named practice_areas and a Consultant table named consultants. Each consultant belongs to exactly one practice area; one practice area can have many consultants. Use an integer primary key for each table, auto-generated IDs, and a foreign key from consultants to practice_areas.

Practice Area fields: id (integer primary key), name (string), floor_number (integer, minimum 1).

Consultant fields: id (integer primary key), name (string), designation (string), consultation_fee (integer, minimum 300), experience_years (integer, minimum 0), practicearea_id (integer foreign key).

Pydantic schemas

- PracticeAreaCreate: name (non-empty string), floor_number (integer >= 1).
- PracticeAreaOut: id, name, floor_number.
- PracticeAreaWithConsultants: id, name, floor_number, and consultants as a list of child output objects.
- ConsultantCreate: name (non-empty string), designation (non-empty string), consultation_fee (integer >= 300), experience_years (integer >= 0). Use this same request schema for create and update.
- ConsultantOut: id, name, designation, consultation_fee, experience_years.

Required endpoints

Method and path	Expected behavior	Success status
POST /practice-areas	Create a Practice Area; return its saved record including generated ID.	201
POST /practice-areas/{parent_id}/consultants	Create a Consultant under the specified parent. Return 404 if parent does not exist.	201
GET /practice-areas/{parent_id}	Return parent details with nested consultants. Optional designation filter uses exact match. Optional sort accepts name or consultation_fee and sorts ascending.	200
PUT /practice-areas/{parent_id}/consultants/{child_id}	Update the child's name, designation, consultation_fee, and experience_years. Do not move it to another parent. Return 404 if parent is missing, child is missing, or child belongs to another parent.	200

Validation and errors

- Return HTTP 422 for an empty required name/designation, missing required body field, invalid type, parent floor/level below 1, fee below 300, or negative experience.
- Return HTTP 404 if the requested parent does not exist.
- For update, return HTTP 404 if the child does not exist or is owned by a different parent.
- If the designation filter is missing or blank, treat it as no filter. If a nonblank value is supplied, match it exactly.
- If sort is supplied with a value other than name or the fee field, return HTTP 404 with a concise detail message.
- Return concise error details; do not silently fall back to a different sort field.

Implementation requirements

- database.py: configure SQLite, create the engine and session factory, and implement get_db as a dependency.
- models.py: define the two SQLAlchemy ORM models and a bidirectional one-to-many relationship.
- schemas.py: define request and response Pydantic models, including nested parent-with-children output.
- main.py: create tables, instantiate FastAPI app, register all four endpoints, and use the get_db dependency for sessions.
- Use ORM queries and commits/refreshes to persist records; do not hard-code records.
- Use if __name__ == '__main__': to run uvicorn on host 0.0.0.0 and port 8080. Importing main.py must not start the server.

Required tests

- Create a parent and verify HTTP 201 and generated ID.
- Create a child under an existing parent; verify HTTP 201 and response fields.
- Create a child under a missing parent; verify HTTP 404.
- Get a parent and verify the children are nested in the response.
- Filter children by exact designation and verify unrelated children are excluded.
- Sort by name and by fee, checking ascending order.
- Send an invalid sort value and verify HTTP 404.
- Update an existing child and verify HTTP 200 and changed fields.
- Attempt update with missing parent, missing child, and child owned by another parent; verify HTTP 404 for each.
- Send invalid body values (empty names, fee below 300, negative experience, invalid floor/level) and verify HTTP 422.
- Use a test database or reset database state so tests do not depend on execution order.

Example request bodies

Create Practice Area:

```
{
  "name": "Data Engineering",
  "floor_number": 3
}
```

Create Consultant:

```
{
  "name": "Alex Morgan",
  "designation": "Senior",
  "consultation_fee": 800,
  "experience_years": 8
}
```

Note: the exercise uses a new business domain, but the technical requirements and edge cases above are mandatory.

Exercise 4 — Trainer and Skill Group API (FastAPI)

A learning company manages skill groups and the trainers assigned to them. Administrators create skill groups, add trainers, retrieve a skill group with its trainers, and update trainer details. Every trainer belongs to one skill group.

Project structure

```
Project/
|– database.py
|– models.py
|– schemas.py
|– main.py
|– requirements.txt
`– tests.py
```

SQLite database: learning.db. Required dependencies: fastapi, uvicorn, sqlalchemy, pydantic, pytest, and httpx for API tests.

Database and relationship

Create a Skill Group table named skill_groups and a Trainer table named trainers. Each trainer belongs to exactly one skill group; one skill group can have many trainers. Use an integer primary key for each table, auto-generated IDs, and a foreign key from trainers to skill_groups.

Skill Group fields: id (integer primary key), name (string), floor_number (integer, minimum 1).

Trainer fields: id (integer primary key), name (string), designation (string), session_fee (integer, minimum 300), experience_years (integer, minimum 0), skillgroup_id (integer foreign key).

Pydantic schemas

- SkillGroupCreate: name (non-empty string), floor_number (integer >= 1).
- SkillGroupOut: id, name, floor_number.
- SkillGroupWithTrainers: id, name, floor_number, and trainers as a list of child output objects.
- TrainerCreate: name (non-empty string), designation (non-empty string), session_fee (integer >= 300), experience_years (integer >= 0). Use this same request schema for create and update.
- TrainerOut: id, name, designation, session_fee, experience_years.

Required endpoints

Method and path	Expected behavior	Success status
POST /skill-groups	Create a Skill Group; return its saved record including generated ID.	201
POST /skill-groups/{parent_id}/trainers	Create a Trainer under the specified parent. Return 404 if parent does not exist.	201
GET /skill-groups/{parent_id}	Return parent details with nested trainers. Optional designation filter uses exact match. Optional sort accepts name or session_fee and sorts ascending.	200
PUT /skill-groups/{parent_id}/trainers/{child_id}	Update the child's name, designation, session_fee, and experience_years. Do not move it to another parent. Return 404 if parent is missing, child is missing, or child belongs to another parent.	200

Validation and errors

- Return HTTP 422 for an empty required name/designation, missing required body field, invalid type, parent floor/level below 1, fee below 300, or negative experience.
- Return HTTP 404 if the requested parent does not exist.
- For update, return HTTP 404 if the child does not exist or is owned by a different parent.
- If the designation filter is missing or blank, treat it as no filter. If a nonblank value is supplied, match it exactly.
- If sort is supplied with a value other than name or the fee field, return HTTP 404 with a concise detail message.
- Return concise error details; do not silently fall back to a different sort field.

Implementation requirements

- database.py: configure SQLite, create the engine and session factory, and implement get_db as a dependency.
- models.py: define the two SQLAlchemy ORM models and a bidirectional one-to-many relationship.
- schemas.py: define request and response Pydantic models, including nested parent-with-children output.
- main.py: create tables, instantiate FastAPI app, register all four endpoints, and use the get_db dependency for sessions.
- Use ORM queries and commits/refreshes to persist records; do not hard-code records.
- Use if __name__ == '__main__': to run uvicorn on host 0.0.0.0 and port 8080. Importing main.py must not start the server.

Required tests

- Create a parent and verify HTTP 201 and generated ID.
- Create a child under an existing parent; verify HTTP 201 and response fields.
- Create a child under a missing parent; verify HTTP 404.
- Get a parent and verify the children are nested in the response.
- Filter children by exact designation and verify unrelated children are excluded.
- Sort by name and by fee, checking ascending order.
- Send an invalid sort value and verify HTTP 404.
- Update an existing child and verify HTTP 200 and changed fields.
- Attempt update with missing parent, missing child, and child owned by another parent; verify HTTP 404 for each.
- Send invalid body values (empty names, fee below 300, negative experience, invalid floor/level) and verify HTTP 422.
- Use a test database or reset database state so tests do not depend on execution order.

Example request bodies

Create Skill Group:

```
{
  "name": "Cloud Computing",
  "floor_number": 3
}
```

Create Trainer:

```
{
  "name": "Alex Morgan",
  "designation": "Senior",
  "session_fee": 800,
  "experience_years": 8
}
```

Note: the exercise uses a new business domain, but the technical requirements and edge cases above are mandatory.

Appendix — Setup, run commands, and self-checklists

Pandas exercises

```
python -m pip install pandas pytest
python main.py
pytest -q
```

Use relative file paths such as data/training_enquiries.csv. The CSV files are included in the ZIP under each exercise folder.

FastAPI exercises

```
python -m pip install fastapi uvicorn sqlalchemy pydantic pytest httpx
python main.py
# or: uvicorn main:app –host 0.0.0.0 –port 8080
pytest -q
```

When testing the API, use FastAPI TestClient or an equivalent HTTP client. Ensure the test database is isolated or reset between tests.

Final readiness checklist

- Can explain the purpose of each function/module without reading the solution.
- Can implement the full pipeline/API from a blank project.
- Can verify exact output columns, types, order, and status codes.
- Can handle duplicates, missing values, unmatched foreign keys, invalid inputs, and missing resources.
- Can explain why inner join removes unmatched rows and why the API checks parent-child ownership during update.
- Can run tests and interpret failures instead of relying only on a successful manual example.